YOLO 目标检测（You Only Look Once）

一、定位与版本谱系

- 1. 核心思想与定位
  - 定义：单阶段（One-Stage）端到端目标检测器，一次前向传播直接回归边界框与类别
  - 关键设计：把检测转化为「网格回归」问题，S×S 网格内每个单元负责预测中心落在其中的目标
  - 与两阶段（Faster R-CNN 系）的区别
    - 两阶段：RPN 生成候选区域 → RoI 分类回归，精度高但慢
    - 单阶段：直接密集预测，速度快，早期小目标弱
  - 与 DETR 系的区别
    - DETR：集合预测 + 匈牙利匹配，天然 NMS-free，但收敛慢、小目标弱
    - YOLO：密集先验 + 标签分配，收敛快，工程生态成熟
  - 核心优势：速度—精度平衡好、部署链路成熟、社区与工具链完善
  - 核心劣势：密集预测依赖 NMS、极端小目标与密集遮挡仍难、版本碎片化
- 2. 版本谱系（务必注意：并非同一团队连续演进）
  - YOLOv1（2016，Joseph Redmon 等）
    - 24 卷积层 + 2 全连接，7×7 网格，输出 S×S×(B×5+C)
    - 单次前向，VOC2007 达 63.4 mAP，45 FPS
    - 局限：小目标差、定位误差大、每格仅预测有限目标
  - YOLOv2 / YOLO9000（2016，Joseph Redmon & Ali Farhadi）
    - 引入 BatchNorm、高分辨率分类器、锚框（K-means 聚类）、Passthrough 细粒度特征
    - Darknet-19 主干；多尺度训练；VOC2007 78.6 mAP
    - YOLO9000：WordTree 联合训练，支持 9000+ 类
  - YOLOv3（2018，Joseph Redmon & Ali Farhadi）
    - Darknet-53 主干；三个尺度预测（FPN 式上采样融合）
    - 分类改用独立 Logistic（支持多标签），COCO 33.0 AP（608）
    - 小目标显著改善，成为工业界长期基线
  - YOLOv4（2020，Alexey Bochkovskiy）
    - CSPDarknet53 + SPP + PANet + Mish 激活
    - CIoU 损失、Mosaic 增强、CutMix、DropBlock、SAT 自对抗训练
    - 提出 Bag of Freebies 与 Bag of Specials 方法论；COCO 43.5 AP（V100 65 FPS）
  - YOLOv5（2020，Ultralytics）
    - 无正式论文，PyTorch 实现，工程化程度最高
    - CSPDarknet + PANet + Focus/S6 结构；自适应锚框、Mosaic、自动学习锚点
    - n/s/m/l/x 五档；一键导出 ONNX/TensorRT/CoreML，生态影响力最大
  - YOLOv6（2022，美团）
    - 面向工业部署：EfficientRep 主干 + Rep-PAN + 解耦头
    - 重参数化（RepVGG 式）、Anchor-free、SimOTA/ATSS 标签分配
    - 自蒸馏训练；n/s/m/l 系列
  - YOLOv7（2022，WongKinYiu 等，即 YOLOv4 团队）
    - E-ELAN 扩展高效层聚合网络；面向拼接结构的模型缩放
    - 辅助头（Aux Head）由粗到细引导；计划化重参数化卷积
    - COCO 51.4 AP / 161 FPS（V100）
  - YOLOv8（2023，Ultralytics）
    - Anchor-free + 解耦头；C2f 模块；TAL 任务对齐分配器
    - 分类用 BCE、回归用 DFL + CIoU
    - 多任务：检测 / 分割 / 姿态 / 分类 / 旋转框（OBB）；API 极简
  - YOLOv9（2024，WongKinYiu 团队）
    - PGI 可编程梯度信息，缓解深层网络信息丢失
    - GELAN 广义高效层聚合网络；v9-C / v9-E 两档
  - YOLOv10（2024，清华大学）
    - 一致双分配（Consistent Dual Assignments）实现 NMS-free 端到端
    - 效率—精度驱动的模型设计：秩引导模块、空间-通道解耦下采样、大核深度卷积、部分自注意力 PSA
    - v10-n 到 v10-x 六档
  - YOLO11（2024，Ultralytics）
    - v8 的迭代升级：C3k2 模块、SPPF、C2PSA 注意力
    - 延续 detect / segment / pose / classify / OBB 全任务线
  - YOLO12（2025，Ultralytics）
    - 注意力为中心的架构（Area Attention）、R-ELAN、FlashAttention 式优化
    - 标注为实验性方向，速度—精度权衡仍在验证
  - 版本命名的坑
    - v4 不是原作者的续作；v5 无论文；v6 是美团；v7/v9 是 v4 团队；v10 是清华
    - 版本号大小 ≠ 论文引用关系，选型看具体实现与部署需求
- 3. 同代与衍生模型（非官方 YOLO 家族）
  - YOLOX（2021，旷视）：Anchor-free + 解耦头 + SimOTA，无锚框范式的重要推手
  - PP-YOLOE / PP-YOLOE+（PaddleDetection）：工业级，支持 TensorRT 部署
  - DAMO-YOLO（阿里达摩院）：MAE-NAS 主干 + RepGFPN + ZeroHead
  - YOLOR：多任务统一表征学习
  - YOLOF：单尺度特征检测，质疑 FPN 必要性
  - Gold-YOLO（华为诺亚）：GD 机制的信息聚合
  - RT-DETR（百度）：非 YOLO，但作为实时端到端检测的主要竞争方案
- 4. 能力对比与选型速览
  - 精度梯队（COCO val2017，640 输入，参考值）：v5x < v7 < v8x ≈ v9 < v10/v11 大模型
  - 速度梯队：n/s 档（v8n / v10n / v11n）适合边缘；x 档适合服务器
  - 轻量化首选：YOLOv8n、YOLOv10n、YOLO11n
  - 精度优先：YOLOv9-E、YOLOv10-X、YOLO11x
  - 端到端无 NMS：YOLOv10、RT-DETR
  - 生态与文档：Ultralytics 系（v5/v8/v11）最完善
- 5. 演进主线：每一版到底解决了什么问题
  - v1：证明「单阶段端到端」可行，速度换精度
  - v2：加 BatchNorm 与锚框，把精度拉回可用区间
  - v3：多尺度 + 残差主干，小目标第一次能用
  - v4：工程技巧集大成，提出 Bag of Freebies / Specials 方法论
  - v5：把训练到部署的链路产品化，生态胜利
  - v6：重参数化 + 自蒸馏，面向工业推理速度
  - v7：架构缩放规律 + 辅助头，精度再上台阶
  - v8：无锚框 + 解耦头 + 多任务统一 API
  - v9：可编程梯度信息，缓解深层信息丢失
  - v10：一致双分配，实现 NMS-free 端到端
  - v11 / v12：工程迭代与注意力化探索
- 6. 论文与代码仓库对照
  - v1：arXiv 1506.02640；作者 Joseph Redmon
  - v2：arXiv 1612.08242（YOLO9000）；作者 Redmon & Farhadi
  - v3：arXiv 1804.02767；作者 Redmon & Farhadi
  - v4：arXiv 2004.10934；作者 Alexey Bochkovskiy 等
  - v5：无论文；仓库 ultralytics/yolov5
  - v6：arXiv 2209.02976；美团视觉团队
  - v7：arXiv 2207.02696；WongKinYiu 等
  - v8：无论文；仓库 ultralytics/ultralytics
  - v9：arXiv 2402.13616；WongKinYiu 团队
  - v10：arXiv 2405.14458；清华大学
  - v11 / v12：无论文；Ultralytics 官方文档与仓库
  - 相关对照：YOLOX arXiv 2107.08430；RT-DETR arXiv 2304.08069

二、核心技术与方法演进

- 1. 整体架构（三段式）
  - Backbone（主干）：特征提取，逐级下采样 8/16/32 倍
  - Neck（颈部）：多尺度特征融合（FPN / PANet / Rep-PAN / GD）
  - Head（头部）：在特征图上预测类别、置信度、框偏移
  - 输出解码：网格坐标 + 锚框 + 步长（Stride）还原到原图尺度
  - 典型超参：输入 640×640，三个预测尺度（80/40/20）
- 2. Backbone 演进
  - Darknet-19（v2）：19 卷积 + 5 池化，轻量高效
  - Darknet-53（v3）：引入残差，53 卷积层
  - CSPDarknet53（v4/v5）：跨阶段部分连接（CSP），减少重复梯度
  - EfficientRep（v6）：RepVGG 式重参数化模块堆叠
  - E-ELAN（v7）：扩展高效层聚合，控制最短最长梯度路径
  - GELAN（v9）：结合 CSPNet 与 ELAN 的广义聚合
  - C2f / C3k2（v8 / v11）：更丰富的梯度流与更少参数
  - 常见算子：Conv+BN+SiLU、Bottleneck、SPPF、Focus（v5 早期）
  - 轻量化替代：MobileNetV3、ShuffleNetV2、GhostNet、EfficientNet-Lite
- 3. Neck 与多尺度融合
  - FPN：自顶向下传递语义
  - PANet：再加自底向上路径，强化定位
  - SPP / SPPF：多尺度池化扩大感受野（v4 / v5）
  - Rep-PAN（v6）：重参数化 PAN
  - BiFPN：加权双向融合（非 YOLO 原生，常被借鉴）
  - GD 机制（Gold-YOLO）：全局信息注入
  - 多尺度策略：3 尺度（P3/P4/P5）为主，大目标场景可加 P6
- 4. Head 与预测形式
  - 耦合头（v3–v7）：分类与回归共享卷积
  - 解耦头（v6/v8+）：分类与回归分支分离，缓解任务冲突
  - Anchor-based（v2–v7）
    - 锚框由 K-means 聚类得到，需匹配先验尺寸
    - 输出 (tx, ty, tw, th) 偏移 + objectness + class
  - Anchor-free（v6/v8/v10+）
    - 直接预测中心点到四边距离（l, t, r, b）或中心点热力图
    - 省去锚框超参，泛化更好
  - 分类头激活：v3+ 用独立 Sigmoid（多标签），v1/v2 用 Softmax
  - 辅助头（v7）：训练时额外监督，推理时移除
- 5. 标签分配（Label Assignment）
  - 静态分配：Max-IoU（v3/v4/v5）、Grid 分配
  - 动态分配
    - ATSS：按统计特性自适应选正样本
    - SimOTA（YOLOX）：把分配建模为最优传输近似
    - TAL（v8）：Task-Aligned Assigner，用分类与回归一致性对齐
    - 一致双分配（v10）：一对多 + 一对一联合，推理时仅用一对一分支实现 NMS-free
  - 正负样本比例与中心先验（center prior）的影响
  - 分配策略直接决定收敛速度与最终精度
- 6. 损失函数
  - 分类损失：BCE（Sigmoid）、Focal Loss（缓解正负失衡）
  - 目标置信度：Objectness BCE
  - 框回归损失
    - IoU → GIoU → DIoU → CIoU（v4 起主流）
    - EIoU、SIoU、WIoU（后续改进）
    - DFL 分布焦点损失（v8）：把框回归变为分布学习
  - 总损失：分类 + 置信度 + 框回归 加权求和
  - 损失权重调节：cls / box / dfl 三项系数
- 7. 后处理与 NMS
  - 标准 NMS：按置信度排序，抑制 IoU 超阈值的框
  - Soft-NMS：以衰减代替直接删除
  - DIoU-NMS：把中心距离纳入抑制判据
  - 类别相关 NMS 与类别无关 NMS
  - NMS-free 方案
    - YOLOv10 一致双分配
    - RT-DETR 集合预测
  - 推理加速：batched NMS、TensorRT 内置 NMS 插件
- 8. 重参数化（Re-parameterization）
  - 思想：训练时多分支增强表达，推理时融合为单路卷积
  - RepVGG Block：3×3 + 1×1 + identity 融合
  - YOLOv6 的 RepBlock、YOLOv7 的计划化重参数化
  - 收益：推理速度提升且不掉精度
  - 代价：训练显存与时间增加，导出后结构不可再训练
- 9. 训练技巧（Bag of Freebies）
  - 数据层：Mosaic、MixUp、CutMix、CopyPaste、随机仿射、HSV 抖动
  - 正则化：DropBlock、权重衰减、标签平滑
  - 优化：SGD/Momentum 或 AdamW、余弦退火、Warmup、EMA 权重平均
  - 训练策略：多尺度训练、Close Mosaic（最后若干轮关闭）、冻结训练
  - 自对抗训练 SAT（v4）
  - 蒸馏：自蒸馏（v6）、教师-学生特征对齐
  - 指数滑动平均 EMA 对最终精度影响显著
- 10. 关键模块速查
  - SPP / SPPF：空间金字塔池化
  - CSP：跨阶段部分连接
  - ELAN / E-ELAN：高效层聚合
  - C2f / C3k2：v8 / v11 的特征复用模块
  - PSA：部分自注意力（v10）
  - Area Attention：区域注意力（v12）
  - RepConv：重参数化卷积
  - DFL：分布焦点损失
  - TAL：任务对齐分配器

三、训练、数据与评估

- 1. 常用数据集
  - COCO 2017：80 类，118K 训练 / 5K 验证，检测标准基准
  - PASCAL VOC：20 类，VOC2007/2012，早期版本主战场
  - Objects365：365 类，大规模预训练常用
  - LVIS：1200 类长尾，考验稀有类
  - Open Images：600 类，含大量层级标注
  - 场景专用
    - VisDrone：无人机视角小目标
    - CrowdHuman：密集行人
    - BDD100K：自动驾驶
    - SKU-110K：货架密集商品
    - xView / DOTA：遥感旋转目标
  - 预训练数据：ImageNet（分类预训练）、Objects365（检测预训练）
- 2. 数据格式与标注
  - YOLO 格式：每行 `class cx cy w h`，坐标归一化到 0–1
  - 目录结构：images/train、images/val、labels/train、labels/val
  - 配置文件：data.yaml（path、train、val、names）
  - 标注工具：LabelImg、Labelme、CVAT、Roboflow、X-AnyLabeling
  - 标注质量要点：框贴合度、漏标、类别混淆、遮挡处理规范
  - 格式转换：COCO JSON ↔ YOLO txt ↔ VOC XML
- 3. 数据增强
  - 几何：随机缩放、平移、旋转、剪切、透视、翻转
  - 颜色：HSV 抖动、亮度对比度、灰度化
  - 组合：Mosaic（4 图拼接）、MixUp、CutMix、CopyPaste
  - 高级：马赛克 9 图、随机擦除、超分增强（小目标）
  - 策略：训练前期强增强、末期关闭 Mosaic 提升收敛稳定性
  - 注意事项：几何增强会改变框尺寸，需同步变换标签
- 4. 训练流程与超参
  - 标准流程：预训练权重加载 → 冻结/解冻 → 主训练 → 微调 → 导出
  - 关键超参
    - imgsz（输入尺寸）、batch、epochs
    - lr0（初始学习率）、lrf（最终学习率比例）、warmup
    - weight_decay、momentum、box/cls/dfl 损失权重
    - 数据增强强度（hsv、degrees、translate、scale、mosaic、mixup）
  - 训练监控：loss 曲线、mAP 曲线、学习率曲线、显存占用
  - 早停与最佳权重保存
  - 多卡训练：DDP；梯度累积应对显存不足
- 5. 评估指标
  - IoU：预测框与真值框交并比
  - AP：单类别 PR 曲线下面积；mAP 为类别平均
  - mAP@0.5：IoU 阈值 0.5（PASCAL 风格）
  - mAP@0.5:0.95：IoU 0.5 到 0.95 步长 0.05 的平均（COCO 主指标）
  - AP_S / AP_M / AP_L：按目标面积分档（COCO 定义）
  - AR：平均召回率，maxDets 取 1/10/100
  - 效率指标：FPS、单帧延迟、参数量 Params、计算量 FLOPs/GFLOPs
  - 部署侧指标：端到端时延（含预处理与 NMS）、显存/内存占用
- 6. 基准结果参考（COCO val2017，640 输入，约值）
  - YOLOv1：VOC2007 63.4 mAP（非 COCO）
  - YOLOv3-608：33.0 AP
  - YOLOv4：43.5 AP，V100 约 65 FPS
  - YOLOv5x：约 50.7 AP；v5s 约 37.4 AP
  - YOLOv7：51.4 AP，V100 约 161 FPS（含后处理）
  - YOLOv8x：约 53.9 AP；v8n 约 37.3 AP
  - YOLOv9-C / v9-E：约 53.0 / 55.6 AP
  - YOLOv10-X：约 54.4 AP（NMS-free）
  - 说明：不同版本的测速条件（是否含 NMS、硬件、TensorRT）差异大，横向对比需统一口径

四、部署与工程落地

- 1. 导出格式
  - ONNX：通用中间格式，跨框架部署首选
  - TensorRT：NVIDIA GPU 最快路径，支持 FP16/INT8
  - OpenVINO：Intel CPU/核显
  - TFLite：移动端与嵌入式
  - CoreML：Apple 生态
  - NCNN / MNN：移动端轻量推理
  - RKNN：瑞芯微 NPU（RK3588 等）
  - Paddle Inference：飞桨生态
  - 导出命令：`yolo export model=yolov8n.pt format=onnx`
- 2. 量化与压缩
  - FP16：几乎无损，GPU 首选
  - INT8 PTQ：需校准集，注意小目标与低置信度掉点
  - INT8 QAT：量化感知训练，精度更好但成本高
  - 剪枝：结构化剪枝、通道剪枝
  - 蒸馏：大模型教师指导小模型
  - 混合精度：敏感层保留 FP16
- 3. 推理优化
  - 预处理加速：letterbox 用 GPU 实现、避免 CPU 拷贝
  - 后处理加速：NMS 上 GPU、批量 NMS、阈值自适应
  - 算子融合：Conv+BN+SiLU 融合
  - 动态/静态 shape 选择：静态 shape 性能更优
  - 批处理与流水线：多流并行、Zero-Copy
  - 端到端时延拆解：预处理 / 推理 / 后处理占比分析
- 4. 硬件平台
  - 服务器 GPU：NVIDIA A100 / H100 / L40S
  - 边缘 GPU：Jetson Orin Nano / NX / AGX
  - 国产 NPU：昇腾 310/910、地平线征程、寒武纪
  - 移动 SoC：高通 Hexagon、苹果 Neural Engine
  - CPU：x86 AVX-512、ARM NEON
- 5. 典型应用场景
  - 自动驾驶：车辆、行人、交通标志、可行驶区域
  - 安防监控：入侵检测、人群计数、遗留物检测
  - 工业质检：缺陷检测、零件计数、装配校验
  - 零售：货架商品识别、自助结算、客流分析
  - 无人机：航拍巡检、农业监测、电力巡线
  - 医疗：病灶检测、细胞计数
  - 体育：球员与球追踪

五、实践与选型

- 1. 选型决策树
  - 要最快上手 + 生态完善：YOLOv8 / YOLO11（Ultralytics）
  - 要 NMS-free 端到端：YOLOv10
  - 要极致精度：YOLOv9-E / YOLOv10-X
  - 要边缘实时：YOLOv8n / YOLOv10n / YOLO11n
  - 要工业级重参数化：YOLOv6
  - 要论文对照基线：YOLOv7 / YOLOX
  - 特殊任务：分割 / 姿态 / OBB 选 Ultralytics 全任务线
- 2. 训练排错清单
  - loss 不降：学习率过大/过小、数据标签错误、类别数与 names 不匹配
  - 训练高测试低：过拟合，加增强、减模型、加数据
  - 全背景预测：正样本过少、锚框尺寸与目标不匹配、未加载预训练
  - 小目标漏检：提高输入分辨率、加 P2 层、用 SAHI 切片推理
  - 密集遮挡漏检：换更大模型、调 NMS 阈值、加 CopyPaste 增强
  - 部署掉点：量化校准不足、预处理不一致（letterbox/归一化）、输出解析错误
  - 显存溢出：降 batch、降 imgsz、开 AMP、梯度累积
- 3. 常见问题与对策
  - 版本混用：不同仓库的权重、配置文件不通用，需对应版本
  - 结果不可复现：随机种子、数据打乱、多卡、cudnn benchmark
  - 数据泄漏：同源图片跨 train/val、视频相邻帧切分
  - 类别不平衡：重采样、类别权重、focal loss
  - 评估口径不一致：是否包含 NMS、是否 multi-scale、是否 TTA
  - 过度调参：先把数据与基线跑通，再谈涨点
- 4. 生态与工具链
  - Ultralytics：v5/v8/v11/v12 统一 API（train / val / predict / export）
  - MMYOLO / MMDetection：OpenMMLab 生态，便于对比实验
  - PaddleDetection：PP-YOLOE 系，国产化部署友好
  - 可视化：Netron（结构）、TensorBoard / W&B（训练）、FiftyOne（数据与预测）
  - 加速推理：TensorRT、ONNX Runtime、OpenVINO、FastDeploy
  - 数据管理：Roboflow、CVAT、Label Studio
- 5. 学习路径建议
  - 阶段 1：跑通官方 demo，理解推理输出格式与后处理
  - 阶段 2：自定义数据集训练，掌握 data.yaml 与增强参数
  - 阶段 3：精读 v1、v3、v5、v8 四篇/四份实现，理解演进主线
  - 阶段 4：手写 NMS 与 IoU 损失，复现一个小规模训练
  - 阶段 5：导出与部署，做一次端到端时延优化
  - 阶段 6：跟踪 v10/v11/v12 的新范式（NMS-free、注意力化）

Presented with xmind